C PROGRAMMING · EXAM-ORIENTED STUDY NOTES

# Other Features and Miscellaneous Functions

atof() · atoi() · atol() · toupper() · tolower() · isalnum() · isalpha() · isdigit() · exit()

### What This Guide Covers

1. Use of `atof()`
2. Use of `atoi()`
3. Use of `atol()`
4. Use of `toupper()`
5. Use of `tolower()`
6. Use of `isalnum()`
7. Use of `isalpha()`
8. Use of `isdigit()`
9. Use of `exit()`
10. Comparison & Revision (tables, FAQs, one-page quick sheet)

Prepared strictly as per the given syllabus topic list — no extra functions added.

# Introduction

The C standard library also provides a set of small but essential **utility functions** used to convert strings to numbers, change character case, classify characters, and terminate a program in a controlled way. This guide covers exactly the nine functions listed in the syllabus: `atof()`, `atoi()`, `atol()`, `toupper()`, `tolower()`, `isalnum()`, `isalpha()`, `isdigit()`, and `exit()`.

> **Quick Header Map:** `atof(), atoi(), atol()` → `<stdlib.h>` | `toupper(), tolower(), isalnum(), isalpha(), isdigit()` → `<ctype.h>` | `exit()` → `<stdlib.h>`

# 1 Use of atof()

## atof() — ASCII TO FLOAT (double)

### Purpose and Definition

`atof()` stands for **"ASCII to Float"**. It converts a **string (character array)** that represents a real number into its equivalent **double** value. It is used whenever numeric data is received as text — for example, from user input, a file, or command-line arguments — and needs to be used in arithmetic calculations.

| | |
|---|---|
| **Header File** | `#include <stdlib.h>` |
| **Return Value** | The converted value as a `double`. Returns `0.0` if no valid conversion could be performed. |

### Converting a String to double — How It Works

The function scans the input string from the beginning, skips leading white-space, optionally reads a `+` or `-` sign, then reads digits, an optional decimal point, more digits, and an optional exponent part (`e` or `E`). Conversion stops as soon as a character is found that cannot be part of a number.

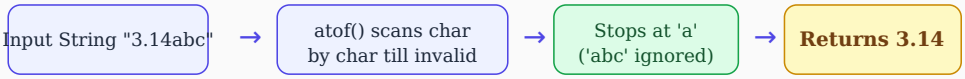



*Fig 2.1 — String-to-number conversion flow used by atof()*

### Step-by-Step Examples

| Input String | Result (double) | Explanation |
|---|---|---|
| `"25.5"` | 25.500000 | Straightforward decimal conversion |
| `" -12.75"` | -12.750000 | Leading spaces skipped, sign handled |
| `"3.14abc"` | 3.140000 | Stops at first non-numeric character 'a' |
| `"3.14abc1"` | 3.140000 | Still stops at 'a' — the '1' after it is never reached |
| `"abc"` | 0.000000 | No valid number found → returns 0.0 |
| `"7"` | 7.000000 | Integer-looking string also converts fine |

**Tricky Point:** Conversion stops **permanently** at the first invalid character — it does not skip over it and resume reading digits later. So `atof("3.14abc1")` is still `3.140000`, not `3.141`.

### C Program with Output

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    char str1[] = "25.75";
    char str2[] = "-9.5abc";
    double val1, val2;

    val1 = atof(str1);
    val2 = atof(str2);

    printf("val1 = %.2f\n", val1);
    printf("val2 = %.2f\n", val2);
```

```
    return 0;
}
```

**OUTPUT**

```
val1 = 25.75 val2 = -9.50
```

## Common Mistakes and Important Exam Points

**Common Mistakes:**

- Forgetting to include `<stdlib.h>`.
- Assuming `atof()` returns an `int` — it always returns `double`.
- Passing a non-numeric string and expecting an error — it silently returns `0.0`, it does **not** raise an error.
- Forgetting to store the result in a `double` variable (storing in `float` may truncate precision).

**Exam Point:** `atof()` takes a `const char*` and always returns `double`, never `float` or `int`. It is declared in `<stdlib.h>`.

# 2 Use of atoi()

## atoi() — ASCII TO INTEGER

### Purpose and Definition

`atoi()` stands for **"ASCII to Integer"**. It converts a string representing whole numbers into an equivalent **int** value. It is the integer counterpart of `atof()` and is commonly used to convert numeric text input (e.g. menu choices, IDs, counts) into usable integers.

| | |
|---|---|
| **Header File** | `#include <stdlib.h>` |
| **Return Value** | The converted value as an `int`. Returns `0` if no valid conversion could be performed. |

### Converting a String to int — How It Works

Just like `atof()`, the function skips leading white-space, reads an optional sign, then reads consecutive digit characters and converts them into their integer value. Conversion stops at the first non-digit character, and anything after it (including a decimal point) is ignored.

### Step-by-Step Examples

| Input String | Result (int) | Explanation |
|---|---|---|
| `"123"` | 123 | Direct conversion |
| `" -45"` | -45 | Leading spaces skipped, sign handled |
| `"56.99"` | 56 | Stops at the decimal point '.' |
| `"12abc"` | 12 | Stops at first letter |
| `"xyz"` | 0 | No digits found → returns 0 |

### C Program with Output

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    char str1[] = "348";
    char str2[] = "-27xyz";
    int num1, num2;

    num1 = atoi(str1);
    num2 = atoi(str2);

    printf("num1 = %d\n", num1);
    printf("num2 = %d\n", num2);

    return 0;
}
```

**OUTPUT**

```
num1 = 348 num2 = -27
```

### Common Mistakes and Important Exam Points

**Common Mistakes:**

- Using `atoi()` on a decimal string and expecting the fractional part to be kept — it is simply truncated at the '.' character.
- Forgetting that an invalid string silently gives `0`, which can be confused with a genuine "0" input.
- Passing an uninitialized or non-null-terminated character array, which causes undefined behaviour.

**Exam Point:** `atoi()` always returns `int` and is declared in `<stdlib.h>`. It does not perform any error checking or exception

handling.

# 3 Use of atol()

## atol() — ASCII TO LONG

### Purpose and Definition

`atol()` stands for **"ASCII to Long"**. It converts a string representing a whole number into an equivalent **long int** value. It works exactly like `atoi()` but is used when the numeric value may be too large to fit inside a normal `int`.

| | |
|---|---|
| **Header File** | `#include <stdlib.h>` |
| **Return Value** | The converted value as a `long int`. Returns `0` if no valid conversion could be performed. |

### Converting a String to long — How It Works

Just like `atoi()`, the function skips leading white-space, reads an optional sign, then reads consecutive digit characters and converts them into their long integer value. Conversion stops at the first non-digit character. The only difference from `atoi()` is the **return type** and the larger range of values it can safely hold.

### Step-by-Step Examples

| Input String | Result (long) | Explanation |
|---|---|---|
| `"100000"` | 100000 | Direct conversion, fits in long easily |
| `"-2000000000"` | -2000000000 | Large negative value beyond typical safe int usage |
| `"45xyz"` | 45 | Stops at first non-digit character |
| `"abc"` | 0 | No digits found → returns 0 |

### C Program with Output

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    char str1[] = "123456789";
    char str2[] = "-987654321";
    long num1, num2;

    num1 = atol(str1);
    num2 = atol(str2);

    printf("num1 = %ld\n", num1);
    printf("num2 = %ld\n", num2);

    return 0;
}
```

**OUTPUT**

```
num1 = 123456789 num2 = -987654321
```

### Common Mistakes and Important Exam Points

**Common Mistakes:**

- Storing the result of `atol()` in a plain `int` variable — this can truncate or overflow large values.
- Using `%d` instead of `%ld` in `printf()` to print a `long` value.
- Forgetting that, like `atoi()`, it gives no error indication for invalid input — it silently returns `0`.

**Exam Point:** `atol()` always returns `long int` and is declared in `<stdlib.h>`. Use it instead of `atoi()` whenever the expected number may exceed the range of a normal `int`.

# 4 Use of toupper()

## toupper() — Convert Character to Uppercase

### Purpose and Definition

`toupper()` converts a single lower-case alphabetic character to its **upper-case** equivalent. If the character passed is not a lower-case letter, it is returned unchanged.

| | |
|---|---|
| **Header File** | `#include <ctype.h>` |
| **Return Value** | Upper-case equivalent of `ch` if it is a lower-case letter; otherwise `ch` is returned unchanged. |

### Character Conversion Process

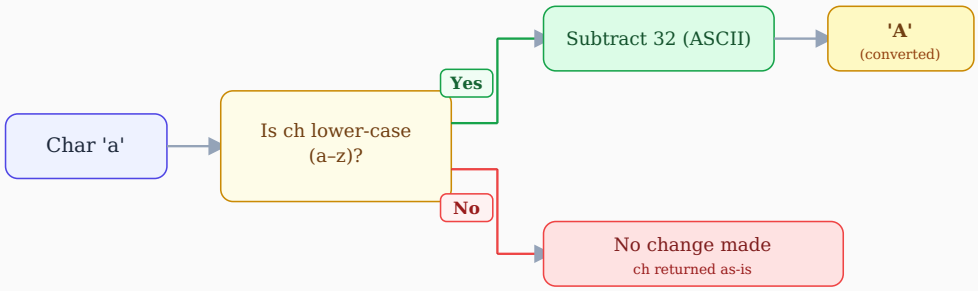



*Fig 5.1 — Character case conversion flow used by toupper() — the "No" branch shows non-lower-case input returned exactly as received*

### Examples with Output

| Input Character | Output |
|---|---|
| 'a' | 'A' |
| 'z' | 'Z' |
| 'M' (already upper) | 'M' (unchanged) |
| '5' (digit) | '5' (unchanged) |

```
#include <stdio.h>
#include <ctype.h>

int main() {
    char ch = 'g';
    printf("Original: %c\n", ch);
    printf("Uppercase: %c\n", toupper(ch));
    return 0;
}
```

**OUTPUT**

```
Original: g Uppercase: G
```

### Common Mistakes and Exam Points

- Believing `toupper()` works on whole strings — it works on **one character at a time**; a string needs a loop.
- Forgetting `<ctype.h>`.
- Assuming it returns `char` — the prototype returns `int`.

**Exam Point:** To convert an entire string, use a loop: `for(i=0;str[i];i++) str[i]=toupper(str[i]);`

# 5 Use of tolower()

## tolower() — Convert Character to Lowercase

### Purpose and Definition

`tolower()` converts a single upper-case alphabetic character to its **lower-case** equivalent. If the character is not an upper-case letter, it is returned unchanged. It is the exact counterpart of `toupper()`.

| | |
|---|---|
| **Header File** | `#include <ctype.h>` |
| **Return Value** | Lower-case equivalent of `ch` if it is an upper-case letter; otherwise `ch` unchanged. |

### Character Conversion Process

Internally, `tolower()` checks whether the character lies in the range `'A'`–`'Z'`. If yes, it **adds 32** to the ASCII value (the reverse of what `toupper()` does) to get the lower-case letter.

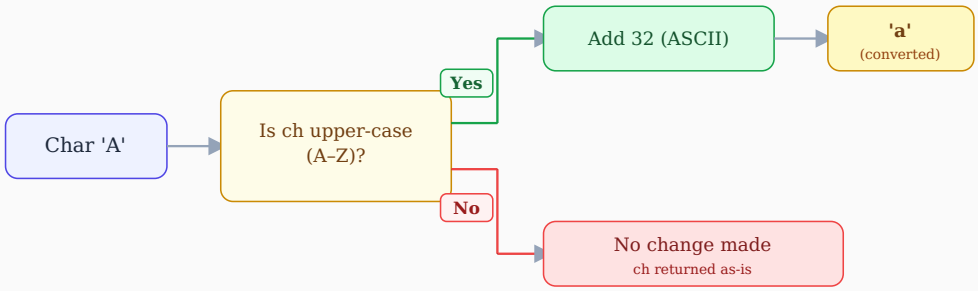



*Fig 6.1 — Character case conversion flow used by tolower() — the "No" branch shows non-upper-case input returned exactly as received*

### Examples with Output

| Input Character | Output |
|---|---|
| 'A' | 'a' |
| 'Z' | 'z' |
| 'k' (already lower) | 'k' (unchanged) |
| '#' (symbol) | '#' (unchanged) |

```
#include <stdio.h>
#include <ctype.h>

int main() {
    char ch = 'R';
    printf("Original: %c\n", ch);
    printf("Lowercase: %c\n", tolower(ch));
    return 0;
}
```

**OUTPUT**

```
Original: R Lowercase: r
```

### Common Mistakes and Exam Points

- Trying to apply it directly to a whole string instead of looping character-by-character.
- Confusing the ASCII shift direction — `tolower()` **adds** 32, `toupper()` **subtracts** 32.
- Forgetting to include `<ctype.h>`, causing implicit-declaration warnings.

**Exam Point:** Both `toupper()` and `tolower()` take and return `int` (not `char`), even though they are used for characters.

# 6 Use of isalnum()

## isalnum() — Check Alphanumeric Character

### Purpose and Definition

`isalnum()` checks whether a given character is **alphanumeric**, i.e. either a letter (A–Z, a–z) or a digit (0–9). It is used to validate user input such as usernames or passwords where only letters and digits are allowed.

| | |
|---|---|
| **Header File** | `#include <ctype.h>` |
| **Return Value** | **Non-zero** (true) if `ch` is a letter or digit; **0** (false) otherwise. |

### Return Behaviour

Like all "is..." functions in `<ctype.h>`, `isalnum()` does **not** return a boolean `1` / `0` in the strict sense — it returns any non-zero value for true, and exactly `0` for false. In practice it is used directly inside `if` conditions.

### Checking Alphanumeric Characters — Examples with Output

| Character | isalnum() Result |
|---|---|
| 'A' | Non-zero (true) |
| '7' | Non-zero (true) |
| '@' | 0 (false) |
| ' ' (space) | 0 (false) |

```
#include <stdio.h>
#include <ctype.h>

int main() {
    char ch1 = 'A', ch2 = '9', ch3 = '#';

    if (isalnum(ch1)) printf("%c is alphanumeric\n", ch1);
    if (isalnum(ch2)) printf("%c is alphanumeric\n", ch2);
    if (!isalnum(ch3)) printf("%c is NOT alphanumeric\n", ch3);

    return 0;
}
```

**OUTPUT**

```
A is alphanumeric 9 is alphanumeric # is NOT alphanumeric
```

**Exam Point:** `isalnum(ch)` is logically equivalent to `(isalpha(ch) || isdigit(ch))`.

# 7 Use of isalpha()

## isalpha() — Check Alphabetic Character

### Purpose and Definition

`isalpha()` checks whether a given character is a **letter of the alphabet** (A–Z or a–z). It ignores digits, spaces, and punctuation.

| | |
|---|---|
| **Header File** | `#include <ctype.h>` |
| **Return Value** | Non-zero (true) if `ch` is an alphabetic letter; `0` (false) otherwise. |

### Return Behaviour

Returns a non-zero value only for the 52 letters (upper- and lower-case). Everything else — digits, symbols, whitespace — produces `0`.

### Checking Alphabetic Characters — Examples with Output

| Character | isalpha() Result |
|---|---|
| 'x' | Non-zero (true) |
| 'Z' | Non-zero (true) |
| '4' | 0 (false) |
| '&' | 0 (false) |

```
#include <stdio.h>
#include <ctype.h>

int main() {
    char ch1 = 'M', ch2 = '3';

    if (isalpha(ch1)) printf("%c is alphabetic\n", ch1);
    if (!isalpha(ch2)) printf("%c is NOT alphabetic\n", ch2);

    return 0;
}
```

**OUTPUT**

```
M is alphabetic 3 is NOT alphabetic
```

**Exam Point:** A common exam program is counting vowels/consonants in a sentence — it always starts by checking `isalpha()` before checking for vowels.

# 8 Use of isdigit()

## isdigit() — Check Numeric Digit

### Purpose and Definition

`isdigit()` checks whether a given character is a **decimal digit** (0–9). It is widely used to validate numeric input character-by-character before using functions like `atoi()`.

| | |
|---|---|
| **Header File** | `#include <ctype.h>` |
| **Return Value** | Non-zero (true) if `ch` is between '0' and '9'; `0` (false) otherwise. |

### Return Behaviour

Only the ten characters `'0'` to `'9'` return a non-zero (true) value. Letters, symbols, and even a minus sign or decimal point return `0`.

### Checking Numeric Digits — Examples with Output

| Character | isdigit() Result |
|---|---|
| '0' | Non-zero (true) |
| '9' | Non-zero (true) |
| 'A' | 0 (false) |
| '.' | 0 (false) |

```
#include <stdio.h>
#include <ctype.h>

int main() {
    char str[] = "A1B2";
    int i, count = 0;

    for (i = 0; str[i] != '\0'; i++) {
        if (isdigit(str[i]))
            count++;
    }
    printf("Number of digits = %d\n", count);

    return 0;
}
```

**OUTPUT**

```
Number of digits = 2
```

### Character Classification — Decision Diagram

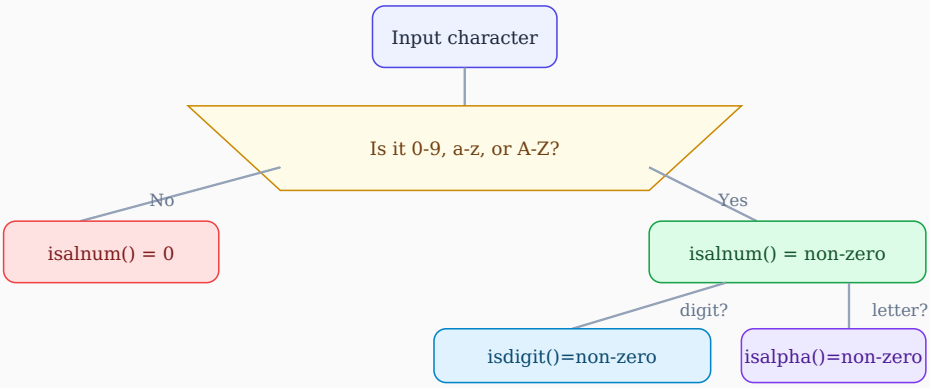



*Fig 9.1 — Character classification decision diagram (isalnum / isalpha / isdigit)*

**Exam Point:** `isdigit()` tests only `'0'`–`'9'` characters, NOT numeric variables. E.g. `isdigit(5)` is different from `isdigit('5')` because it tests the ASCII value 5, not the character '5' (ASCII 53).

# 9 Use of exit()

## exit() — Terminate Program Execution

### Purpose and Definition

`exit()` immediately **terminates the entire program**, no matter which function it is called from. It is used to end a program early — for example, when an unrecoverable error occurs, or when a menu-driven program's "Exit" option is chosen.

| | |
|---|---|
| **Header File** | `#include <stdlib.h>` |
| **Return Value** | None — `exit()` never returns to the caller; the program ends. |

### How Program Execution Terminates

When `exit()` is called: (1) any open files are flushed and closed, (2) any functions registered with `atexit()` are executed, (3) control is returned to the operating system along with the status code. Execution does **not** return to `main()` or to the point where `exit()` was called — unlike a normal `return`, it can be called from **any function**, at **any depth** of nested calls.

### Return / Status Value

| Status Code | Meaning |
|---|---|
| `EXIT_SUCCESS` (usually 0) | Program terminated normally / successfully |
| `EXIT_FAILURE` (usually 1) | Program terminated due to an error |

### Program Termination Flow using exit()

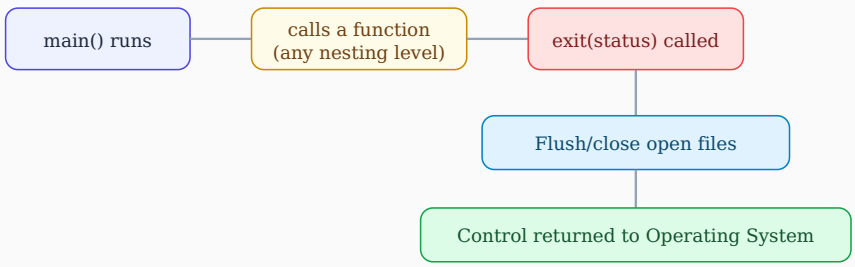



*Fig 10.1 — Program termination flow using exit()*

### Examples with Output

```c
#include <stdio.h>
#include <stdlib.h>

void checkValue(int n) {
    if (n < 0) {
        printf("Error: Negative value not allowed!\n");
        exit(EXIT_FAILURE);   /* program ends here */
    }
    printf("Value is valid: %d\n", n);
}

int main() {
    printf("Program started\n");
    checkValue(-5);
    printf("This line will NEVER execute\n");
    return 0;
}
```

**OUTPUT**

```
Program started Error: Negative value not allowed!
```

**Note:** The line `"This line will NEVER execute"` is not printed because `exit()` terminated the program from inside `checkValue()`, before control could return to `main()`.

### Important Exam Points

- `exit()` can be called from **any function**, not just `main()`.
- It requires `<stdlib.h>`.
- `EXIT_SUCCESS` and `EXIT_FAILURE` are macros defined in `<stdlib.h>` (values 0 and 1 respectively on most systems).
- Unlike `return`, `exit()` does not "return" a value to a calling function — it hands a status code to the operating system.

# Comparison & Revision

### atof() vs atoi() vs atol()

| Point | atof() | atoi() | atol() |
|---|---|---|---|
| Full form | ASCII to Float | ASCII to Integer | ASCII to Long |
| Return type | `double` | `int` | `long` |
| Handles decimal point | Yes | No (stops at '.') | No (stops at '.') |
| Use case | Real / decimal numbers | Normal-range whole numbers | Very large whole numbers |
| Header file | <stdlib.h> | <stdlib.h> | <stdlib.h> |
| Invalid input result | 0.0 | 0 | 0 |

### toupper() vs tolower()

| Point | toupper() | tolower() |
|---|---|---|
| Converts | lower-case → UPPER-case | UPPER-case → lower-case |
| ASCII operation | Subtracts 32 | Adds 32 |
| Non-letter input | Returned unchanged | Returned unchanged |
| Header file | <ctype.h> | <ctype.h> |

### isalnum() vs isalpha() vs isdigit()

| Point | isalnum() | isalpha() | isdigit() |
|---|---|---|---|
| Tests true for | Letters + digits | Letters only (A-Z, a-z) | Digits only (0-9) |
| Relationship | isalpha() OR isdigit() | Subset of isalnum() | Subset of isalnum() |
| Header file | <ctype.h> | <ctype.h> | <ctype.h> |
| Return value | Non-zero / 0 | Non-zero / 0 | Non-zero / 0 |

### return vs exit() (where directly relevant)

| Point | return | exit() |
|---|---|---|
| Scope | Ends only the current function | Ends the entire program immediately |
| Usable in | Any function; in main() it ends the program too | Any function, at any call depth |
| Value passed | Returns a value to the caller function | Passes a status code to the Operating System |
| Header required | None (language keyword) | <stdlib.h> |

### Function Syntax and Return-Value Summary Table

| Function | Header File | Prototype | Return Value |
|---|---|---|---|
| atof() | <stdlib.h> | `double atof(const char*);` | double value / 0.0 on failure |
| atoi() | <stdlib.h> | `int atoi(const char*);` | int value / 0 on failure |
| atol() | <stdlib.h> | `long atol(const char*);` | long value / 0 on failure |
| toupper() | <ctype.h> | `int toupper(int);` | Uppercase char / unchanged |
| tolower() | <ctype.h> | `int tolower(int);` | Lowercase char / unchanged |
| isalnum() | <ctype.h> | `int isalnum(int);` | Non-zero / 0 |
| isalpha() | <ctype.h> | `int isalpha(int);` | Non-zero / 0 |
| isdigit() | <ctype.h> | `int isdigit(int);` | Non-zero / 0 |
| exit() | <stdlib.h> | `void exit(int);` | None — terminates program |

## Frequently Asked Exam Questions

**Q1. What is the difference between atoi() and atof()?**
A. atoi() converts a string to int and stops at a decimal point; atof() converts a string to double and correctly reads the decimal part. Both are in <stdlib.h>.

**Q2. What header files are required for toupper(), tolower(), isalpha(), isdigit(), isalnum()?**
A. All five belong to <ctype.h>.

**Q3. What does isalnum() check?**
A. It checks whether a character is either a letter (A-Z, a-z) or a digit (0-9); it returns non-zero for true and 0 for false.

**Q4. Can exit() be called from a function other than main()?**
A. Yes. exit() can be called from any function at any point in the program and will terminate execution immediately, regardless of call depth.

**Q5. What is the difference between return and exit()?**
A. return exits only the current function (and ends the program only if called from main()); exit() terminates the whole program instantly from anywhere and hands a status code to the OS.

**Q6. What value do atoi(), atof() and atol() return for an invalid (non-numeric) string?**
A. atoi() returns 0; atol() returns 0; atof() returns 0.0 — no error is raised in any case.

**Q7. What is the significance of EXIT_SUCCESS and EXIT_FAILURE?**
A. They are predefined macros in <stdlib.h> (usually 0 and 1) passed to exit() to indicate normal or abnormal termination to the operating system.

**Q8. When should atol() be used instead of atoi()?**
A. atol() should be used when the numeric string may represent a value larger than the range of a normal int, since it returns a long int instead.

## Common Errors and Misconceptions

- Thinking `toupper()` / `tolower()` operate on whole strings directly (they only work on one character; strings need a loop).
- Forgetting the correct header: `<stdlib.h>` for atof/atoi/atol/exit, `<ctype.h>` for the case and classification functions.
- Assuming the "is..." functions return strictly `1` for true — they return **any non-zero value**.
- Expecting `atoi()` / `atof()` / `atol()` to detect and report invalid input — they simply return 0 / 0.0 silently.
- Using `%d` instead of `%ld` when printing the `long` result of `atol()`.
- Believing code written after an `exit()` call in the same function will still run — it will not.

## One-Page Quick Revision Sheet

| Function | Header | Converts / Checks | Returns |
|---|---|---|---|
| atof() | stdlib.h | string → double | double value (0.0 if invalid) |
| atoi() | stdlib.h | string → int | int value (0 if invalid) |
| atol() | stdlib.h | string → long | long value (0 if invalid) |
| toupper() | ctype.h | lower char → upper | upper char / unchanged |
| tolower() | ctype.h | upper char → lower | lower char / unchanged |
| isalnum() | ctype.h | letter or digit? | non-zero / 0 |
| isalpha() | ctype.h | letter only? | non-zero / 0 |
| isdigit() | ctype.h | digit only? | non-zero / 0 |
| exit() | stdlib.h | terminates program | none (hands status to OS) |

**Remember:** stdlib.h → atof, atoi, atol, exit | ctype.h → toupper, tolower, isalnum, isalpha, isdigit. All "is..." functions return non-zero (true) or 0 (false) — never assume exactly 1. atoi()/atof()/atol() never throw errors on bad input; they silently return 0 / 0.0. exit() can be called from anywhere and stops the program immediately — code after it never runs.